

SDPO: Reinforcement Learning ผ่านการทำ Self-Distillation

จุดจบของคอขวดข้อมูล 1-Bit
สู่ยุคของการใช้ Rich Feedback
ในการฝึกสอน Language Models

Student Policy

Self-Teacher
Policy (SDPO)

Retrospective Mirror

ปัญหาหลัก: คอขวดข้อมูลของ RLVR

```
1 import osop
2 import con-5ene.
3 traceback (naet reall last):
4   File "pytnons.py", line 26, in <module>
5     main Toan > reovesucc():
6     safe+erover+e oeroot
7   File "pytonon.py", line 23, in <module>
8     ceon = ceon()
9     return 8
10  File "/Users/weatoitn/scenreeste.py", line 58, in <module>
11    rotoannae(cveeandsvetobe)
12  File "/ueers/uantaria/sconreeta.py", line 18, in main
13    retun = cobve.foncktesee(roasal)
14  File "/haers/weatoile/sconreecta.py", line 13, in lewsn
15    return orros(roae)
16 Stack Srrrov: Eneertoe-stack Errors
17 procttesioop()
18
```



• RLVR
(Verifiable Rewards)

การสูญเสียข้อมูล

โมเดลในปัจจุบันเรียนรู้ผ่านผลลัพธ์แบบ
Scalar (เช่น ผ่าน = 1, ไม่ผ่าน = -1 หรือ 0)

ปัญหาเครดิต (Credit Assignment)

โมเดลรู้แค่ว่า “ทำงานพลาด” แต่ไม่รู้ว่า
“พลาดตรงไหน” หรือ “ทำไมถึงพลาด”
ทำให้การเรียนรู้ชะงักเมื่อเจองานที่ซับซ้อน

การยกระดับสู่ **RLRF** (Reinforcement Learning with Rich Feedback)

RLVR (Verifiable Rewards)



RLVR

Feedback = Scalar Reward (-1 หรือ 1)

เครดิต = สุ่มให้ทั้งประโยค (Sparse)

ความเข้าใจ = รู้แค่ผิดพลาด (That it went wrong)

RLRF (Reinforcement Learning with Rich Feedback)

Runtime Error: ZeroDivisionError, Line 73
Tracaback (most recent call last):
File "/users/example/code.py", line 73, in calculate_average
return total / count



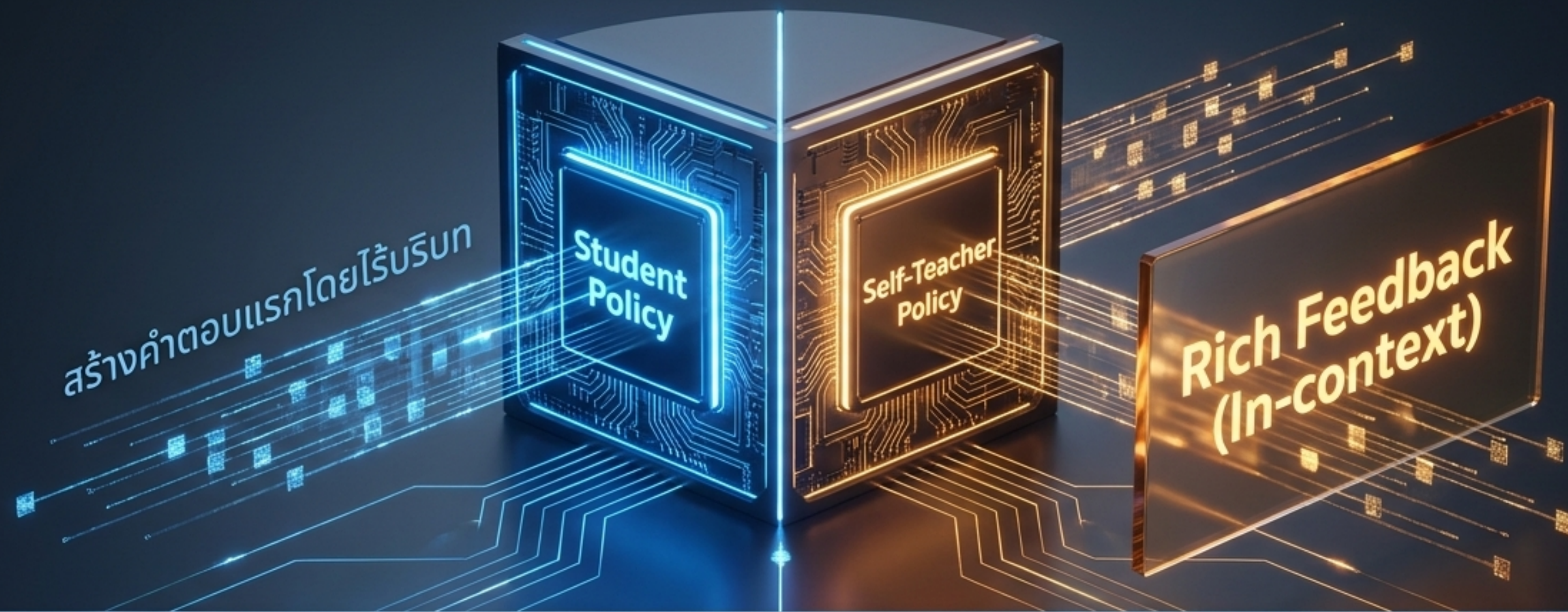
RLRF

Feedback = Tokenized Text (เช่น Runtime Error)

เครดิต = เจาะจงแต่ละคำ (Dense)

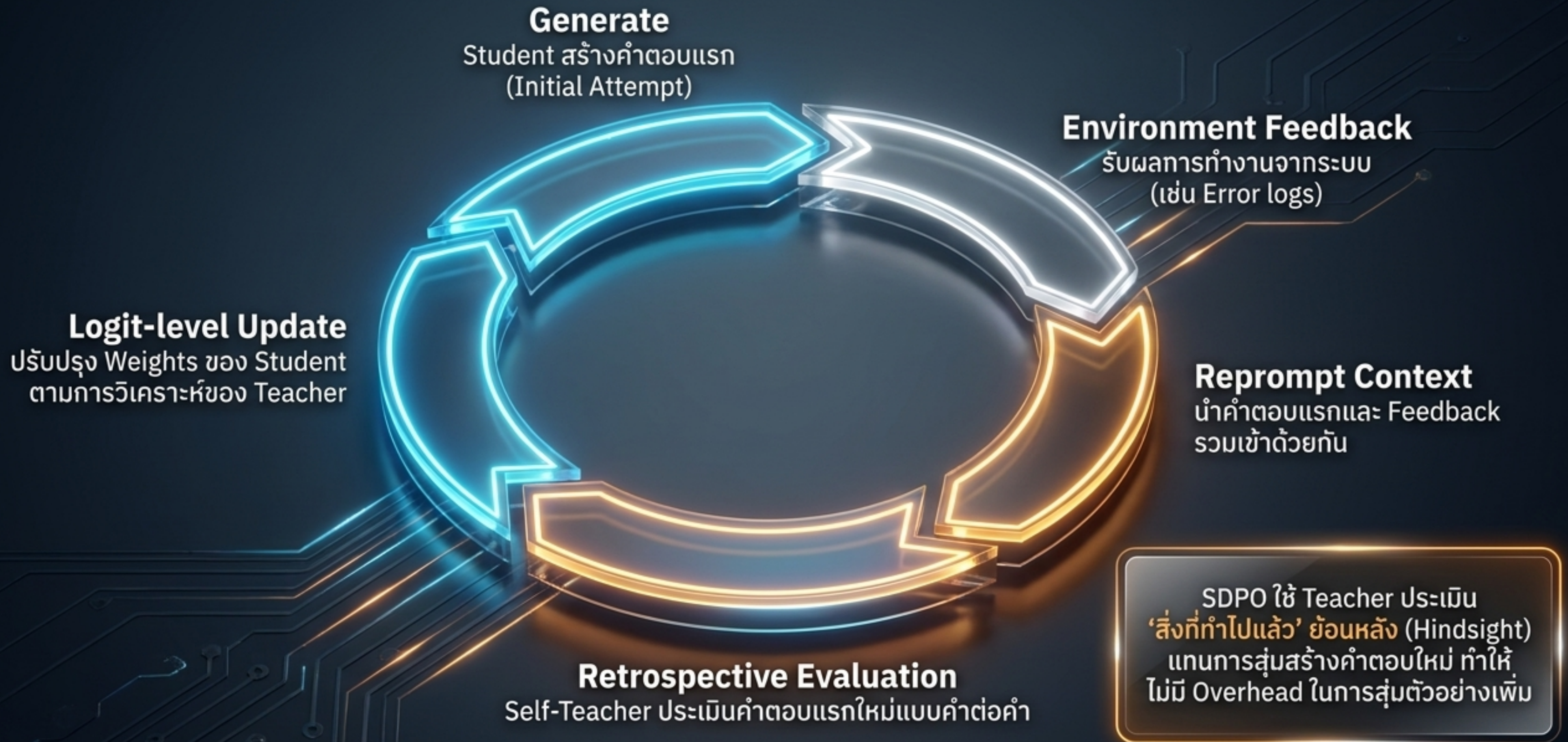
ความเข้าใจ = รู้ว่าผิดที่ไหนและอย่างไร (What went wrong)

แนวคิดหลัก: LLM ในฐานะ ‘ครู’ ของตัวเอง



Language Models มีความสามารถ In-context learning ที่ทรงพลังอยู่แล้ว
เมื่อเราป้อนข้อผิดพลาด (Feedback) กลับเข้าไปใน Context โมเดลเดียวกันนี้จะสามารถวิเคราะห์หาข้อ
บกพร่องและแก้ไขแนวทางของตัวเองได้โดยไม่ต้องพึ่งพาโมเดลที่เก่งกว่า (External Teacher)

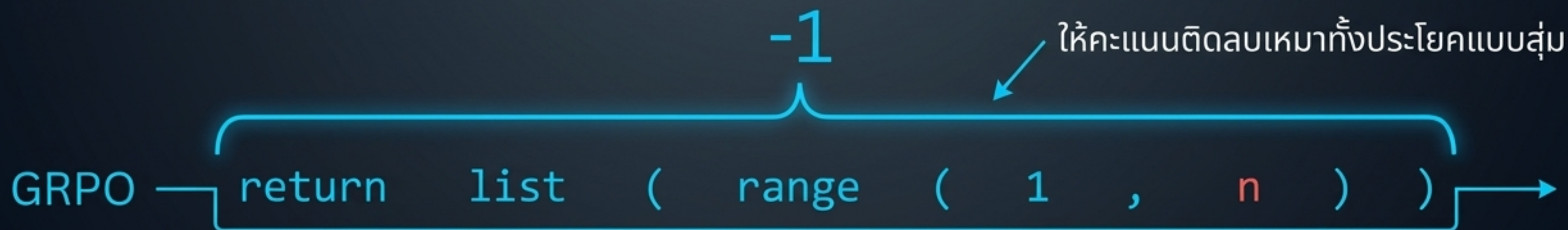
กลไกการทำงานของ SDPO (Self-Distillation Policy Optimization)



พัฒนาการของภูมิทัศน์ Post-Training

	เรียนรู้ระหว่างการทำงานจริง (On-Policy Learning)	ใช้ประโยชน์จากข้อมูลเชิงลึก (Rich Signal Usage)	ไม่ต้องพึ่งพาโมเดลที่เก่งกว่า (External Teacher Independence)
Supervised Fine-Tuning (SFT)	✗	✓	✗
On-Policy Distillation	✓	✓	✗
RLVR (เช่น GRPO)	✓	✗	✓
SDPO (วิธีการของเรา)	✓	✓	✓

Dense Credit Assignment: ความแม่นยำระดับ Logit



SDPO ให้ Feedback เฉพาะในระดับคำ (**Logit-level**) ทำให้โมเดลเรียนรู้ได้อย่างละเอียดว่าคำไหนควรเก็บไว้ และคำไหนคือต้นเหตุของข้อผิดพลาดอย่างแท้จริง

บทพิสูจน์ 1: เหนือกว่าแม่ในมาตรฐานเดิม (RLVR Environments)

Chemistry Benchmark



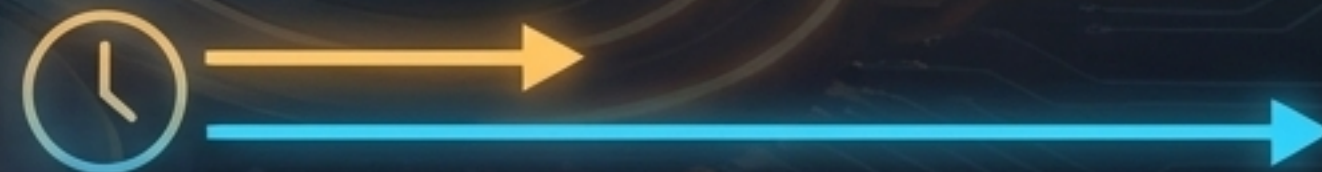
Physics Benchmark



ชนะ GRPO อย่างชัดเจน:
ความแม่นยำ 70.2% เทียบกับ 66.6%

อย่าง GRPO ความแม่นยำ 70.2% เทียบกับ 66.6% ↑

ความเร็วในการเรียนรู้: SDPO ใช้เวลาเพียง **1 ชั่วโมง** เพื่อให้ได้ประสิทธิภาพเทียบเท่า GRPO ที่ฝึกสอนนานถึง 5 ชั่วโมง (เร็วกว่าสูงสุด 6 เท่า)



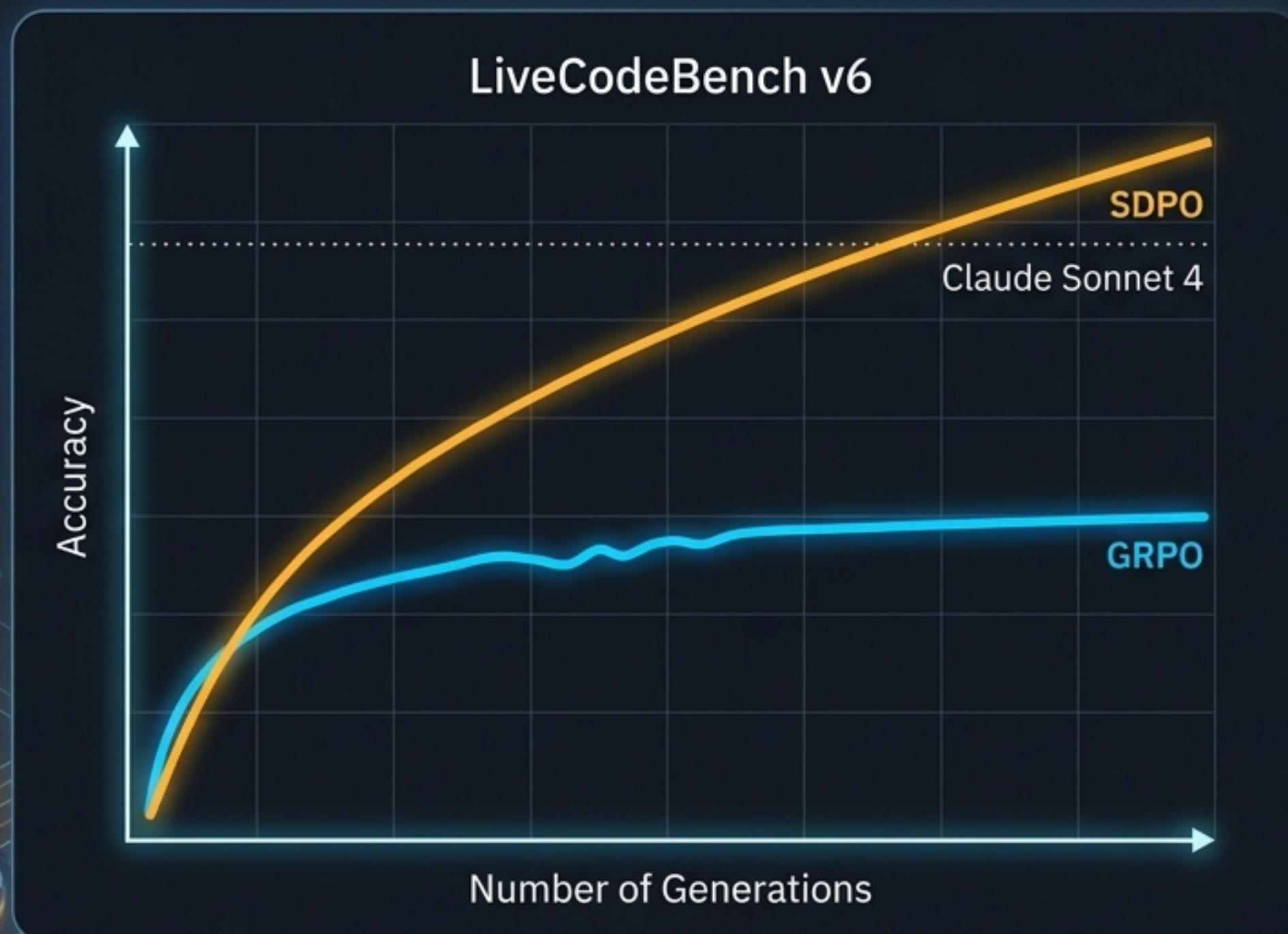
โดยใช้ ‘คำตอบที่ถูกของตัวเองในรอบนั้น’ เป็น **Feedback แบบนัย (Implicit)**
SDPO ก็สามารถเอาชนะ GRPO ได้โดยไม่ต้องปรับแก้ Environment

ทลายขีดจำกัดด้านประสิทธิภาพ: การให้เหตุผลที่กระชับ



เครดิตระดับคำที่แม่นยำของ SDPO ช่วยจัดการคิดแบบ 'พายุเรือในอ่าง' (Circular reasoning) พิสูจน์ให้เห็นว่า Language Models ไม่จำเป็นต้องใช้ความยาว (Verbosity) เสมอไปเพื่อให้ได้เหตุผลที่ถูกต้อง

บทพิสูจน์ 2: การเอาชนะด้วย **Rich Feedback** ทางโปรแกรมมิ่ง



ทำลายสถิติ:

แม่นยำถึง 48.8% เทียบกับ 41.2% (GRPO)

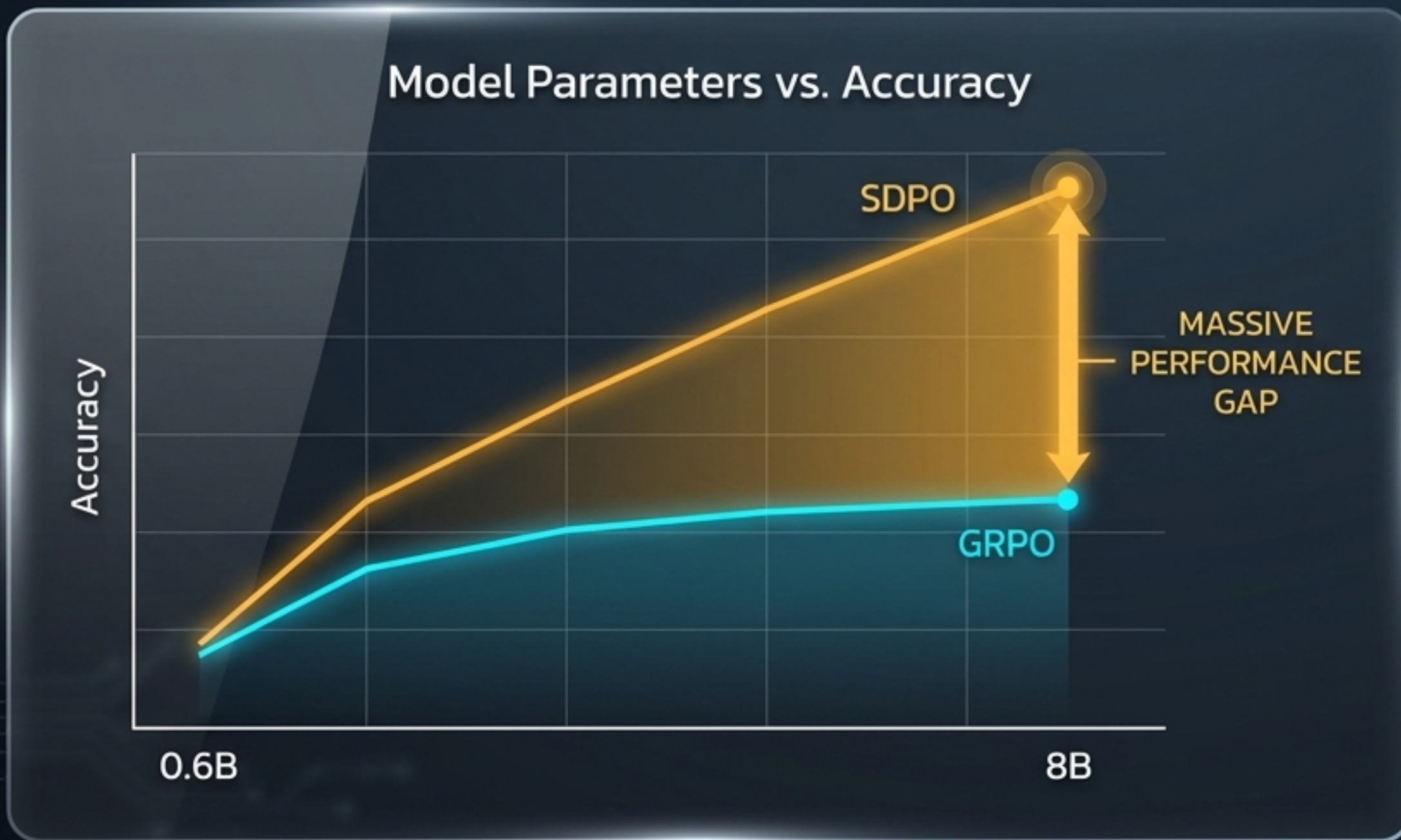
ประสิทธิภาพด้าน Data:

ถึงจุดสูงสุดของ GRPO โดยใช้การ Generation น้อยกว่าถึง 4 เท่า

เหนือกว่า SOTA:

ชนะ instruct models ระดับแนวหน้าอย่าง Claude Sonnet 4 บน LiveCodeBench v6

กฎการขยายตัวของการเรียนรู้ด้วยตนเอง (Scaling Law of Self-Correction)



ความสามารถ **In-context learning** ของโมเดลจะก้าวกระโดดเมื่อขนาดใหญ่ขึ้น

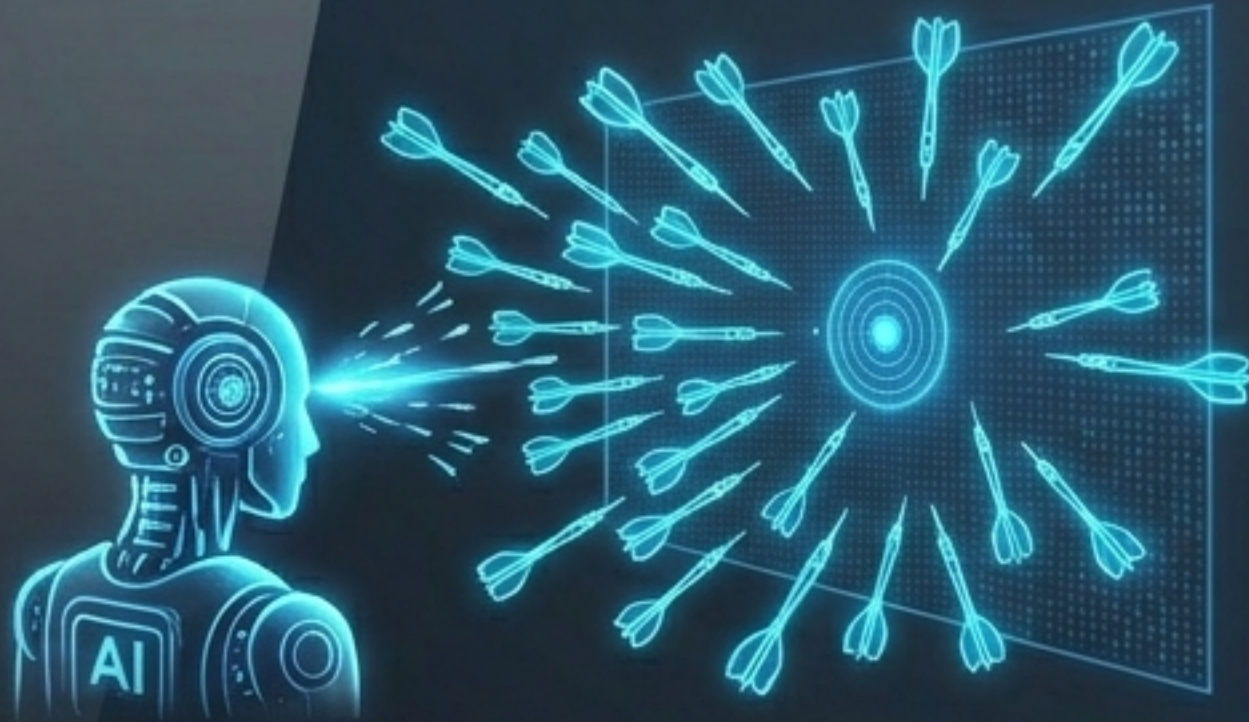
ดังนั้น 'ครู' (Self-Teacher) จึงฉลาดขึ้นตามไปด้วย

ผลลัพธ์: ส่วนต่างประสิทธิภาพของ SDPO จะยิ่งทิ้งห่างคู่แข่งมากขึ้น ในยุคของ Frontier Models ที่มีพารามิเตอร์มหาศาล

พรมแดนใหม่: ข้อจำกัดของการแก้ปัญหาซับซ้อนที่ Test-Time

Problem Schematic

Best-of-K



สุ่มหาคำตอบซ้ำๆ แต่ ‘ไม่เคยเรียนรู้’
จากข้อผิดพลาดเลย

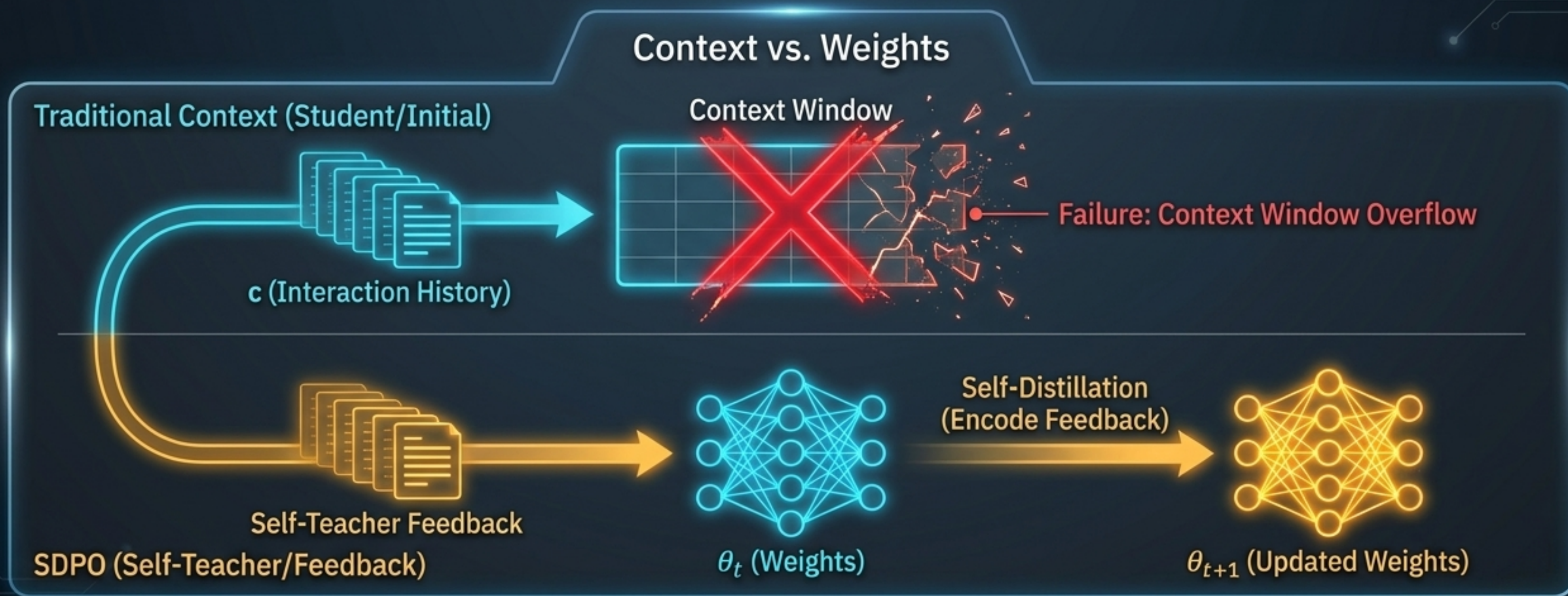
Multi-Turn Prompting



ป้อน **Feedback** กลับเข้า **Context** แบบสนทนา
แต่สะสมข้อมูลจนล้นความจุของ Context Window

คำถามที่ท้าทายที่สุดที่มีคำตอบแทนแบบ 0/1 (Binary Reward) ไม่สามารถแก้ได้ด้วยวิธีดั้งเดิมเหล่านี้

การบีบอัด Context ลงสู่ค่าน้ำหนัก (Weights) โดยตรง



แทนที่จะต่อท้ายประวัติการสนทนาล้น SDPO ใช้ Self-Distillation แบบ batch size = 1 เพื่อ 'เข้ารหัส' (Encode) วิธีแก้ปัญหาจาก Feedback ลงไปในพารามิเตอร์ของโมเดล ทำให้ข้ามขีดจำกัดเรื่องความยาว Context ไปได้อย่างสิ้นเชิง

บทพิสูจน์ 3: การเร่งความเร็วการค้นพบคำตอบ (Test-Time Discovery)



ข้อสรุปประสิทธิภาพ (Efficiency Summary)

บนโจทย์ที่ยากระดับสุดยอด SDPO
สามารถค้นพบคำตอบได้เร็วกว่า
Best-of-K และ Multi-turn ถึง **3 เท่า**

ตัวอย่างจริง (Real-World Example)

แก้โจทย์ที่วิธีอื่นไม่สามารถหาคำ
ตอบได้เลยใน 2,750 ครั้ง โดย
SDPO ใช้เพียง **321 ครั้ง**

อนาคตของ Post-Training และการคิดทบทวนของ AI



SDPO พิสูจน์ให้เห็นว่า Language Models ไม่ได้ต้องการแค่เวลาในการคิดวิเคราะห์ที่นานขึ้น—แต่พวกมันต้องการวิธีการสะท้อนและทบทวนข้อผิดพลาดที่ลึกซึ้งขึ้นต่างหาก

Train-Time

เรียนรู้ตรงจุด เลิกคิดวนเวียน
นำไปสู่ความแม่นยำสูงด้วยข้อมูลที่น้อยกว่า

Test-Time

ยุคใหม่ของการทำ Test-Time Scaling
ที่เปลี่ยนข้อผิดพลาดให้เป็นองค์ความรู้ถาวรใน Weights